

Document Analysis & Steganography

Office files · metadata · macros · hidden messages

Stefanus Eko Prasetyo

Lab-3 · hands-on session

```
student@dfir:~/lab03$  
$ olevba YearlyBonus.docm  
    VBA MACRO ThisDocument.cls  
$ zsteg starry_night.png  
    b1,rgb,lsb,xy .. text:
```

Today's roadmap

1

Documents as evidence

what a file carries beyond what you see on screen

2

Office file formats

OLE vs OOXML - and why a docx is secretly a ZIP

3

Metadata

exiftool: authors, timestamps, and GPS coordinates

4

Macros

oletools: detecting and reading VBA payloads

5

Steganography

PNG least-significant bits, JPEG steghide, audio spectrograms

6

Challenges & working paper

upstream cases + two new challenges + documentation rules

Every file is a layered witness

What you see is one layer. A document also carries **metadata, embedded code, and hidden payloads** - and in a forensic setting, those layers often matter more than the visible text.

Visible layer

the letter, the slide, the photo - what the author meant you to see

Metadata

creator, editing time, GPS of the camera - what the software recorded for free

Embedded code

VBA macros that run on open - the classic phishing payload

Hidden payloads

steganography: secrets folded into pixels, coefficients, or frequencies

OLE vs OOXML

	OLE (1997-2003)	OOXML (2007-now)
Extensions	.doc .ppt .xls .rtf	.docx .pptx .xlsx (+m for macros)
Structure	binary 'file within a file' container	ZIP archive of XML parts
Read with	oletools, structured parsers	unzip + any text/XML tool
Forensic angle	macros in binary VBA streams	everything is inspectable - text, rels, props

Key idea: an OOXML file is a ZIP. Rename report.docx to report.zip, extract, and read the document as text - no Office license, no risk of executing anything.

A docx is a ZIP - open it like one

```
student@dfir:~/lab03$  
$ cp Doc1.docx work/ && cd work      # work on a copy  
$ unzip Doc1.docx  
  inflating: [Content_Types].xml  
  inflating: _rels/.rels  
  inflating: word/document.xml  
  inflating: word/_rels/document.xml.rels  
  inflating: docProps/core.xml  
  inflating: docProps/app.xml  
  inflating: word/styles.xml  word/settings.xml  
# visible text lives in word/document.xml
```

Why investigators care

- ▶ no macros execute while you unzip
- ▶ hidden text stays in the XML even if Word does not show it
- ▶ timestamps and author survive in docProps
- ▶ grep works: strings, grep, xmllint - the whole Linux toolbox

The parts that matter to us

Part	Holds	Investigator's question
<code>docProps/core.xml</code>	title, author, created/modified dates	who wrote it, and when?
<code>docProps/app.xml</code>	application, template, company	what software produced it?
<code>word/document.xml</code>	the actual text - including HIDDEN text	is there more than we can see?
<code>word/_rels/*.rels</code>	relationships: styles, images, EXTERNAL links	does it phone home?
<code>word/vbaProject.bin</code>	compiled VBA macros (.docm/.xlsm/.pptm)	what code runs on open?

exiftool — the metadata goldmine

```
student@dfir:~/lab03$  
$ exiftool image1.jpg  
Artist           : Michael Scott  
Software         : Picasa  
Date/Time Original: 2013:04:27 15:32:09  
GPS Position      : 34 deg 12' 39.00" N,  
                  118 deg 26' 11.00" W  
  
$ exiftool -gps:all -a image1.jpg # just GPS  
# 34.2109, -118.4364 -> Burbank, Los Angeles
```

What metadata answers

- ▶ who made it - Author, Artist, Creator fields
- ▶ when - created vs modified (backdating leaves gaps)
- ▶ where - GPS latitude/longitude from cameras and phones
- ▶ works on images, audio, video, AND Office files

Macro-enabled documents (.docm)

A macro is a program inside the document. Attackers love it because **one click on 'Enable Content' executes their code** with the user's privileges.

Auto-run hook	When it fires
<code>AutoOpen()</code> / <code>Document_Open()</code>	when the document is OPENED in Word
<code>AutoClose()</code> / <code>Document_Close()</code>	when it is closed - wipes tracks after exfiltration
<code>Workbook_Open()</code>	Excel's version, on spreadsheet open
<code>Shell</code> / <code>WScript.Shell</code>	classic giveaway: launching external processes

Macros are stored compiled in word/vbaProject.bin - you need a VBA parser to read them, not a text editor.

oletools: oleid first, then olevba

```
student@dfir:~/lab03$  
  
$ sudo -H pip install -U oletools  
$ oleid HelloWorld.docm  
File format      : MS Word 2007+ Macro-Enabled  
Encrypted        : False  
VBA Macros      : Yes    Risk: Medium  
External Relationships : 0  
  
$ olevba HelloWorld.docm  
VBA MACRO NewMacros.bas  
Sub HelloWorld() ... ' full source code
```

Reading order

- ▶ oleid = triage: format, encryption, macros, external rels
- ▶ olevba = extraction: full VBA source + suspicious keywords
- ▶ olevba flags IOC keywords: Shell, CreateObject, Hex strings...
- ▶ mraptor: quick verdict if a macro is malicious

Hidden writing - and why we chase it

Cryptography scrambles a message so it cannot be read; steganography hides it so it cannot be **seen to exist**. The carrier looks completely innocent.

440 BC

Herodotus: a message tattooed on a shaved scalp, revealed when the hair grew back

Invisible ink

lemon juice on paper, revealed by heat - the classroom classic

Malicious memes

2018: Stegoloader malware fetched commands hidden in Twitter images

Forensic stakes

exfiltrated data and C2 channels can ride inside cat photos

PNG — messages in the least significant bit

```
student@dfir:~/lab03$  
$ zsteg starry_night.png  
b1,rgb,lsb,xy .. text: "148:The fishermen know  
    that the sea is dangerous and the storm terrible,  
    but they have never found these dangers sufficient  
    reason for remaining ashore."  
b2,r,lsb,xy      .. file: OpenPGP Public Key  
  
# pixel (200,50): R=52 G=171 B=226 -> 110100 10 101010 11 111000 10  
# flip the final bit of each channel: invisible to the eye
```

Reading zsteg output

- ▶ b1 = bit plane 1 (the LSB); rgb = channels scanned; xy = row-major order
- ▶ zsteg brute-forces many plane/channel combinations at once
- ▶ PNG is lossless - LSBs survive saving intact
- ▶ install: `sudo gem install zsteg`

JPEG — steghide and the pair-swapping trick

```
student@dfir:~/lab03$  
  
# hide a secret (attacker side)  
$ steghide embed -cf cover.jpg -ef secret.txt  
Enter passphrase: *****  
  
# extract (investigator side)  
$ steghide extract -sf cover.jpg  
Enter passphrase:  
wrote extracted data to "secret.txt".  
# try an EMPTY passphrase first - many samples use one
```

Why JPEG differs from PNG

- ▶ JPEG is lossy - re-encoding destroys LSBs
- ▶ steghide swaps DCT coefficient pairs instead
- ▶ embedding often needs a passphrase
- ▶ combine with exiftool: metadata may hold the hint

Spectrograms — reading sound as a picture

A spectrogram plots **frequency (vertical) against time (horizontal)**. If someone 'draws' with specific frequencies, the drawing becomes readable text.

- 1 Open the .wav in Audacity
- 2 Click the track's drop-down menu
- 3 Choose Spectrogram view
- 4 Read the bright shapes - they spell the message

```
student@dfir:~/lab03$  
# alternate route: render it yourself  
$ sox super_secret_audio.wav -n  
    spectrogram -o spec.png  
$ eog spec.png      # then just read it  
# LSB audio stego also exists - spectrogram first,  
# bit-plane tools second
```

The document & stego toolbox

Tool	Purpose	Install
oleid	triage an Office file: format, encryption, macros, external rels	<code>pip install oletools</code>
olevba	extract full VBA source + suspicious keyword analysis	<code>pip install oletools</code>
exiftool	read/write metadata: author, dates, GPS - images, audio, docs	<code>apt install libimage-exiftool-perl</code>
zsteg	detect PNG LSB steganography (bit planes, channels, orders)	<code>gem install zsteg</code>
steghide	embed/extract in JPEG, BMP, WAV (DCT pair swapping)	<code>apt install steghide</code>
Audacity	visualize audio spectrograms	<code>apt install audacity</code>

Your challenges

Challenge A

upstream case files

YearlyBonus.docm + Presentation.pptx
+ .wav

- ▶ decode a macro's secret
- ▶ unmask a mole via metadata
- ▶ read a spectrogram flag
- ▶ solved together in the Deep Dive

from the original lab

Challenge B — new

cover_letter.docx

files/cover_letter.docx

- ▶ a job application arrives
- ▶ looks clean on screen...
- ▶ the XML tells a longer story
- ▶ OOXML: parts, rels, hidden text

hidden text hunt

Challenge C — new

harbor_photo.png

files/harbor_photo.png

- ▶ a stock harbor photo
- ▶ metadata says 'nothing here'
- ▶ zsteg disagrees
- ▶ recover the drop-note and flag

LSB hunt

Flag format: `ctf{xxxx}` (Challenge A keeps the upstream flag format)

Working paper & evidence rules

1 Pre-lab first

section 1.4 of the working paper, before the session

2 Work on copies

copy the docm/pptx/png into a work folder - never open originals

3 Never 'Enable Content'

analyzing a macro file means reading it, not running it

4 Number your evidence

screenshots and outputs get B-01, B-02, C-01 in the evidence log

5 Hash what you touch

integrity form before and after analysis

```
student@dfir:~/lab03$  
  
# evidence log entry  
[B-03] 2026-10-07 09:41  
      grep -o 'vanish' document.xml  
      6 hidden runs found  
      -> text hidden from screen view
```

No evidence, no answer.

every claim must trace back to a numbered exhibit

KEY TAKEAWAYS

01

A docx is a ZIP

unzip it, grep it, read the XML - hidden text and metadata included

02

Macros are code, not content

oleid to triage, olevba to read - never 'Enable Content' on evidence

03

Metadata convicts

exiftool turns Artist fields and GPS tags into names and places

04

Stego hides in the noise

zsteg for PNG LSBs, steghide for JPEG, spectrograms for audio

Open the file. Read the layers.



student@dfir:~/lab03\$

```
$ git clone https://github.com/stprasst/digital-forensic-lab
$ cd digital-forensic-lab/Lab\ 03/files
$ ls
YearlyBonus.docm  Presentation.pptx  super_secret_audio.wav
starry_night.png  cover_letter.docx  harbor_photo.png
```